

SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE
FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL



Design de Classes - Casos de Uso

CSU06 - Realizar Inscrição

CSU09 - Criar Turma

Luiz Felipe Coutinho Bouchardet Sant'Anna - 5256

Florestal - MG

Setembro de 2025

Lógica de telas iniciais

Tela 1 - Inicial:

Tela inicial com um botão apenas JOGAR.

Ao clicar no botão JOGAR abre uma nova “janela falsa” com duas novas opções:

- NOVO JOGO: Ao clicar em novo jogo a “janela falsa” atualiza para uma tela de login para o jogador inserir seu nickname.
- CONTINUAR JOGO: Ao clicar em continuar jogo a “janela falsa” atualiza para uma tela de login.
- TURMAS: Ao clicar em gerenciar turmas a “janela falsa” atualiza para uma tela para o professor gerenciar as funções de turmas.

Tela 2 - Novo jogo:

A tela para criação de um novo jogador contará com:

- Campo para digitar o nickname do jogador.
- Campo para digitar a senha do jogador.
- Campo para digitar a turma que o jogador será inserido.
- Botão de CRIAR JOGADOR, para confirmar a criação e iniciar o jogo.
- Botão de VOLTAR, para voltar à tela anterior.

Tela 3 - Continuar jogo:

A tela para login de um jogador já existente contará com:

- Campo para digitar o nickname do jogador.
- Campo para digitar a senha do jogador.
- Botão de LOGIN, para confirmar o login.
- Botão de VOLTAR, para voltar à tela anterior.

Tela 4 - Gerenciar turmas:

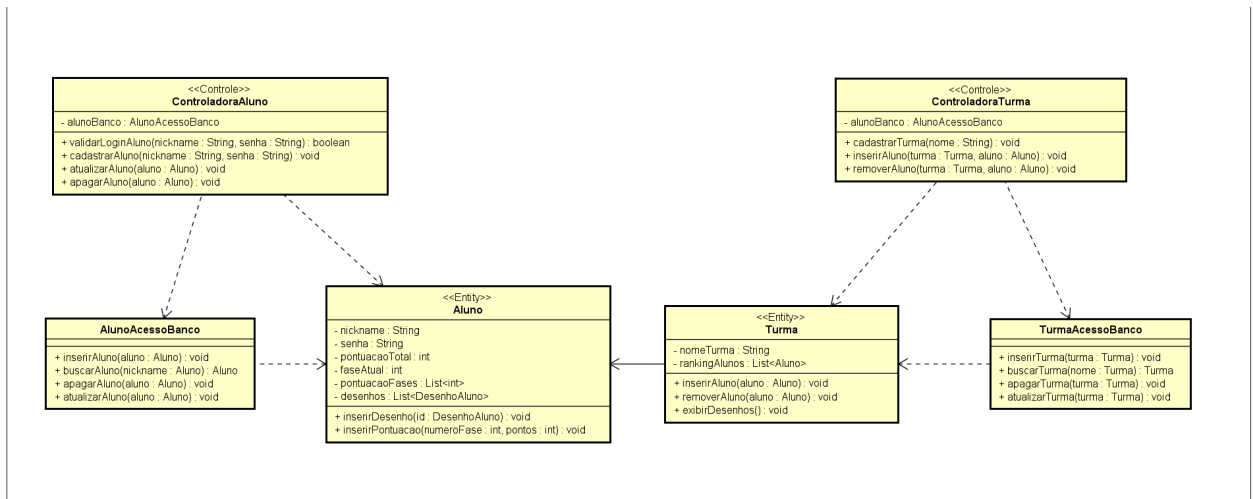
A tela para gerência de turmas possui algumas funcionalidades:

- Botão para CRIAR NOVA TURMA, onde o professor adiciona uma nova turma ao banco.
- Botão para VER GALERIA DE TURMAS, onde o professor consegue verificar rankings e imagens criadas pelos alunos, por turma.
- Botão de VOLTAR, para voltar à tela anterior.

Observações:

- Um jogador terá um login para uma conta, de forma com que ele consiga continuar seu progresso em tentativas distintas, com suas estatísticas e progresso salvo.
- O professor não terá inscrição, apenas criará uma turma para futuras verificações. Alunos também poderão verificar facilmente a galeria de turmas apenas acessando as funcionalidades de turmas.

Diagrama de classes



Explicação de classes e métodos

Foi pensada uma lógica para separação de responsabilidade das classes, nos casos de uso sendo implementados, para a criação ou login de um jogador, e a criação de turmas. Foram criadas 3 classes para cada uma das opções, sendo elas uma classe controladora (ControladoraAluno e ControladoraTurma), que ficará com a responsabilidade de gerenciar as modificações para estas duas classes, com seus métodos realizando as chamadas de métodos internos necessários das classes.

Também foram criadas duas classes de acesso ao banco de dados (AlunoAcessoBanco e TurmaAcessoBanco), que ficarão com a responsabilidade de manipular todos os dados inseridos e removidos do banco.

Por fim as duas classes de entidade, Aluno que possui um conjunto de pontuações para cada fase, uma pontuação total, e um conjunto de ids para seus desenhos criados, tal qual como seu nickname, senha e fase atual (progresso no jogo). Para a classe Turma foi pensado uma implementação simples de nome e uma lista de alunos, que se manterá sempre ordenada de acordo com a pontuação total dos jogadores, se mantendo como o ranking da turma.

Aluno:

- `inserirDesenho`: recebe um inteiro correspondente ao id de um desenho criado, e adiciona na lista de desenhos do jogador.
- `inserirPontuacao`: recebe como parâmetro um valor de pontuação como inteiro, e a fase respectiva. Caso a fase esteja zerada em `pontuacaoFases`, adiciona pontuação na posição correspondente, caso já exista uma pontuação permanece a maior pontuação da fase. Atualiza a pontuação total, somando com a nova pontuação.

ControladoraAluno:

- `validarLoginAluno`: recebe nickname e senha, chama a função `buscarAluno` no banco, verifica se o nickname existe, caso verdadeiro, verifica se a senha corresponde ao aluno e retorna verdadeiro para senha correta.
- `cadastrarAluno`: recebe nickname e senha, verifica se o nickname já existe, caso não exista

(NULL) cria um novo aluno com os parâmetros recebidos. E outros atributos de aluno com valores iniciais.

- atualizarAluno: recebe um Aluno, verifica a existência de seu nickname no banco e atualiza seus valores.
- apagarAluno: recebe um Aluno e apaga ele do banco de dados.

AlunoAcessoBanco:

- inserirAluno: recebe como parâmetro um Aluno e insere ele no banco de dados.
- buscarAluno: recebe como parâmetro um nickname e caso exista retorna um Aluno, caso não exista retorna NULL.
- apagarAluno: recebe como parâmetro um Aluno e apaga do banco de dados.
- atualizarAluno: recebe como parâmetro um Aluno e atualiza suas informações no banco de dados.

Turma:

- inserirAluno: insere o aluno no ranking, na posição correta de acordo com sua pontuação total, um novo jogador sempre é adicionado ao final da lista.
- removerAluno: encontra o aluno no ranking e o remove..
- exibirDesenhos: percorre o ranking de alunos, exibindo os desenhos de cada um, para a galeria de desenhos.

ControladoraTurma:

- cadastrarTurma: recebe nome da turma, chama a função buscarTurma no banco, verifica se a turma existe, caso negativo cria uma nova turma.
- inserirAluno: recebe uma turma e um aluno, chama a função inserirAluno, de Turma, para adicionar o aluno na turma.
- removerAluno: recebe uma turma e um aluno, chama a função removerAluno, de Turma, para remover o aluno da turma.

TurmaAcessoBanco:

- `inserirTurma`: recebe como parâmetro uma Turma e insere ela no banco de dados.
- `buscarTurma`: recebe como parâmetro um nickname e caso exista retorna uma Turma, caso não exista retorna NULL.
- `apagarAluno`: recebe como parâmetro uma Turma e apaga do banco de dados.
- `atualizarAluno`: recebe como parâmetro uma Turma e atualiza suas informações no banco de dados.

Observações:

- Alunos e suas informações serão salvos no banco de dados.
- Alunos têm uma lista de imagens, composta pelos objetos `DesenhoAlunos`, com as informações de suas imagens construídas.
- Desenhos criados por alunos serão salvos no banco de dados, desenhos fixos das fases apenas em arquivos de assets do jogo.